FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

FECAP

CENTRO UNIVERSITÁRIO ÁLVARES PENTEADO

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ANNA PAULA ALVES SILVA

DILLY MARTINS DA SILVA

LAURA RAYSSA SOUZA ARAÚJO

RAFAELA CARVALHO BARCOS MELLO

Mini-relatório 4U

Professor: Eduardo Savino Gomes.

São Paulo

2026

1. Introdução

A 4U é uma aplicação web responsiva para o acompanhamento escolar bimestral no Ensino Fundamental, com três perfis: Administrador, Professor e Pai/Responsável. O professor registra, para cada aluno, a média do bimestre, tags de acompanhamento e uma descrição. O registro passa pela revisão da administração e, depois de publicado, chega ao responsável, que dá ciência da leitura e gera o PDF. O problema resolvido é a comunicação fragmentada entre escola e famílias. O back-end usa Node.js, Express e MySQL com Programação Orientada a Objetos, e o front-end usa React. Este relatório mostra onde as estruturas de dados aparecem no código e como foram usadas para tornar a aplicação mais eficiente.

2. Estruturas de dados aplicadas

A análise do código mostrou que a classe *Sistema* não guarda listas em memória: alunos, professores e acompanhamentos ficam no MySQL, e a classe executa as regras de negócio em transações. As estruturas de dados aparecem em quatro pontos.

2.1 Dicionário (tabela de dispersão) e grafo: fluxo de status

Onde: *TRANSICOES_VALIDAS*, na classe *Acompanhamento (classes/Acompanhamento.js)*. **Por quê:** o registro segue o fluxo Rascunho → Enviado → Em revisão → Publicado, com Devolvido e Cancelado. Cada status é uma chave do objeto, e o valor é a lista dos próximos status permitidos. O objeto JavaScript funciona como uma tabela de dispersão (*hash*), em que o acesso pela chave é O(1). **Problema resolvido:** toda mudança de status (enviar, revisar, devolver, publicar) é validada em um só lugar, sem uma cadeia de if/else. Uma transição inválida devolve o erro HTTP 409.

2.2 Map com lista de tempos (janela deslizante): limite de tentativas

Onde: *middlewares/limitarTentativas.js*, usado nas rotas de login e de "Esqueci minha senha". **Por quê:** um Map associa cada IP (chave) a um vetor com os horários das tentativas recentes. Assim, a consulta e a gravação por IP custam O(1), sem percorrer os registros de todos os IPs. **Problema resolvido:** barra ataques de força bruta (no máximo 10 logins a cada 10 minutos por IP), sem consultar o banco (trecho 4.2).

2.3 Conjunto (Set): remoção de duplicatas

Onde: *[...new Set(b.tags.map(Number))]* em *controllers/acompanhamentoController.js* e *[...new Set(l.disciplinas.split('|'))]* em *controllers/cadastroController.js*. **Por quê:** o Set só guarda valores únicos e faz inserção e verificação em O(1). **Problema resolvido:** se a mesma tag for enviada

duas vezes, ela não é gravada em duplicidade. Um professor que leciona a mesma disciplina em várias turmas tem essa disciplina listada uma só vez (trecho 4.3).

2.4 Árvore B+ (índices do MySQL) e paginação

Onde: *Banco de Dados/02_indices.sql* cria 7 índices, como *idx_aluno_nome* e *uq_usuario_email*. A listagem de cadastros usa *LIMIT/OFFSET (utils/formato.js)*. **Por quê:** no InnoDB, os índices são árvores B+ ordenadas. O login busca o e-mail em O(log n) e a busca pelo início do nome *(LIKE 'Ana%')* aproveita o índice. A paginação traz para a aplicação só 10, 20 ou 50 itens por vez. **Problema resolvido:** as telas continuam rápidas com o banco de demonstração completo (120 alunos e quase 4 mil acompanhamentos).

3. Otimização: abordagem escolhida × alternativa

Ponto	Alternativa menos eficiente	Abordagem do projeto	Ganho prático
Validar status	Vetor de pares [de, para] percorrido a cada mudança: O(t)	Dicionário por status: O(1) + lista curta (≤ 3)	Regra em um único lugar; tempo constante
Limite de login	Vetor de objetos {ip, tempos} com find: O(k), k = IPs	Map IP → tempos: O(1)	Custo não cresce com o número de visitantes
Tags sem repetição	includes dentro de um laço: O(n²)	Set: O(n)	Nenhuma tag gravada em dobro
Login por email	Varredura da tabela inteira: O(n)	Índice único (árvore B+): O(log n)	Resposta estável mesmo com muitos usuários

Ponto de melhoria identificado. Em *painelController.js* (função do painel do professor), as contagens de cada vínculo são obtidas com *contagens.filter(...)* dentro de *vinculos.map(...)*. Isso dá custo O(V × C), em que V é o número de vínculos e C o número de linhas de contagem. Com poucos vínculos por professor o efeito é pequeno, mas a melhoria realista é agrupar as contagens em um Map antes do laço, o que reduz o custo para O(V + C) (trecho 4.3). O mesmo padrão aparece no relatório do responsável, em *tags.filter* por item. Outros dois limites: a busca por sobrenome *(LIKE '% Silva%')* não usa o índice, e um índice *FULLTEXT* resolveria isso; o *Map* de tentativas nunca apaga IPs antigos, e uma limpeza periódica das chaves com vetor vazio evitaria o crescimento da memória.

4. Trechos de código

4.1 Dicionário de transições

Arquivo Backend/src/classes/Acompanhamento.js , classe Acompanhamento

```
const TRANSICOES_VALIDAS = {
  RASCUNHO:   [STATUS.ENVIADO, STATUS.CANCELADO],
  ENVIADO:    [STATUS.EM_REVISAO, STATUS.CANCELADO],
  EM_REVISAO: [STATUS.PUBLICADO, STATUS.DEVOLVIDO, STATUS.CANCELADO],
  DEVOLVIDO:  [STATUS.ENVIADO, STATUS.CANCELADO],
  PUBLICADO:  [],
  CANCELADO:  []
};

_transicionar(novoStatus) {
  const permitidos = TRANSICOES_VALIDAS[this._status] || [];
  if (!permitidos.includes(novoStatus)) {
    throw new ErroApi(409, `Não é possível passar de "${ROTULOS[this._status]}" ...`);
  }
  this._status = novoStatus;
}
```

O objeto representa o grafo do fluxo como lista de adjacência: cada chave (vértice) aponta para o vetor de destinos permitidos. *TRANSICOES_VALIDAS[this._status]* é um acesso direto por chave, em O(1). Em seguida, *includes* percorre no máximo 3 elementos. Os métodos *enviarParaRevisao(), devolver() e publicar()* chamam *_transicionar*, de modo que uma nova regra de fluxo exige alterar apenas o dicionário.

4.2 Map com janela deslizante

Arquivo *Backend/src/middlewares/limitarTentativas.js* , função *limitarTentativas*

```
const registros = new Map();
const janela = janelaMinutos * 60 * 1000;

return (req, res, next) => {
  const agora = Date.now();
  const chave = req.ip;
  const recentes = (registros.get(chave) || []).filter((t) => agora - t < janela);
  if (recentes.length >= maximo) return next(new ErroApi(429, mensagem));
  recentes.push(agora);
  registros.set(chave, recentes);
  next();
};
```

registros.get e *registros.set* são operações O(1) do Map. O vetor de cada IP funciona como uma fila de horários: o *filter* descarta as tentativas fora da janela, e o *push* inclui a atual no fim. Como o vetor nunca passa de *maximo* elementos, o custo por requisição é constante.

4.3 Sugestão de melhoria (ainda não aplicada ao projeto)

```
// Agrupa uma vez: O(C)
const porVinculo = new Map();
for (const c of contagens) {
  const chave = `${c.turma_id}-${c.disciplina_id}`;
  if (!porVinculo.has(chave)) porVinculo.set(chave, []);
  porVinculo.get(chave).push(c);
}
// Cada vínculo busca o seu grupo em O(1): total O(V + C)
const doVinculo = porVinculo.get(`${v.turma_id}-${v.disciplina_id}`) || [];
```

A chave composta (turma + disciplina) substitui o *filter* repetido. Cada linha de contagem é lida uma única vez, e cada vínculo acessa o seu grupo diretamente pelo *Map.*

5. Conclusão

No projeto 4U, as estruturas de dados foram usadas nos pontos em que fazem diferença. Dicionários e *Map* dão acesso direto por chave no fluxo de status e no controle de tentativas, o *Set* garante valores únicos, e os índices em árvore B+ do MySQL, junto com a paginação, mantêm as consultas rápidas com milhares de registros. A análise também mostrou trechos que ainda podem ser otimizados, como o *filter* dentro de um laço no painel do professor. O principal aprendizado do grupo foi escolher a estrutura pela operação mais frequente (buscar por chave, evitar repetição, ordenar) e avaliar a complexidade antes de escrever o laço, e não depois que a aplicação fica lenta.